Batch Code: ANP_D1838

Student ID: AF05202193

Student Name: S. Vaishnavi

LAB-08

Question:

1. Explain about encapsulation? Write a program and following comments.

2. Explain about abstraction? Write a program and following comments.

---

1. ENCAPSULATION:

Encapsulation in Java means wrapping data (variables) and methods (functions) together inside a single unit (class) and restricting direct access to the data. It is achieved by declaring variables as private and providing public getter and setter methods for controlled access. This ensures data hiding, security, and maintainability.

- Declare variables as private.
- Provide public getters and setters to access or modify them.

Binding data and methods into one unit (class) and hiding internal details from outside access.

PROGRAM:

Declare variables as private = Emp.java

```
public class Emp {
	private int id;
	private String name;
	private String dept;
	private int salary;
	private String city;
	public int getId() {
		return id;
```

```
	}
	public String getName() {
		return name;
	}
	public String getDept() {
		return dept;
	}
	public int getSalary() {
		return salary;
	}
	public String getCity() {
		return city;
	}
	public void setId(int id) {
		this.id = id;
	}
	public void setName(String name) {
		this.name = name;
	}
	public void setDept(String dept) {
		this.dept = dept;
	}
	public void setSalary(int salary) {
		this.salary = salary;
	}
	public void setCity(String city) {
		this.city = city;
	}
}
```

Emp_Details.java

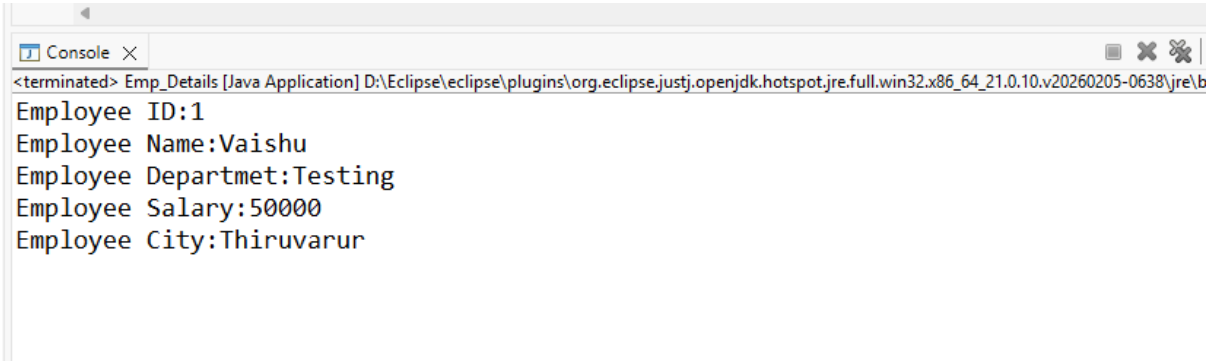
```
public class Emp_Details {

	public static void main(String[] args) {
		// TODO Auto-generated method stub
		Emp emp = new Emp();
```

```
            emp.setId(1);
            emp.setName("Vaishu");
            emp.setDept("Testing");
            emp.setSalary(50000);
            emp.setCity("Thiruvarur");

            System.out.println("Employee ID:" + emp.getId());
            System.out.println("Employee Name:" + emp.getName());
            System.out.println("Employee Departmet:" + emp.getDept());
            System.out.println("Employee Salary:" + emp.getSalary());
            System.out.println("Employee City:" + emp.getCity());
        }
}
```

OUTPUT:



COMMENTS:

Emp.java

1. Class Declaration

- public class Emp - Defines a class named Emp.
- File name must match class name - Emp.java.

2. Private Variables (Data Hiding)

- private int id; - Employee ID (hidden from outside).

- private String name; - Employee Name.
- private String dept; - Employee Department.
- private int salary; - Employee Salary.
- private String city; - Employee City.

3. Getter Methods (Read Access)

- public int getId() - Returns employee ID.
- public String getName() - Returns employee name.
- public String getDept() - Returns employee department.
- public int getSalary() - Returns employee salary.
- public String getCity() - Returns employee city.

4. Setter Methods (Write Access)

- public void setId(int id) - Assigns value to employee ID.
- public void setName(String name) - Assigns value to employee name.
- public void setDept(String dept) - Assigns value to employee department.
- public void setSalary(int salary) - Assigns value to employee salary.
- public void setCity(String city) - Assigns value to employee city.

Emp_Details.java

1. Class Declaration

- public class Emp_Details - Defines main class.
- File name must match class name - Emp_Details.java.

2. Main Method

- public static void main(String[] args) - Entry point of program.
- Program starts execution here.

3. Object Creation

- Emp emp = new Emp(); - Creates an object emp of class Emp.

- This object is used to access getters and setters.

4. Setting Values (Using Setters)

- emp.setId(1); - Sets employee ID = 1.
- emp.setName("Vaishu"); - Sets employee name = Vaishu.
- emp.setDept("Testing"); - Sets employee department = Testing.
- emp.setSalary(50000); - Sets employee salary = 50000.
- emp.setCity("Thiruvarur"); - Sets employee city = Thiruvarur.

5. Getting Values (Using Getters)

- System.out.println("Employee ID:" + emp.getId()); - Prints employee ID.
- System.out.println("Employee Name:" + emp.getName()); - Prints employee name.
- System.out.println("Employee Departmet:" + emp.getDept()); - Prints employee department.
- System.out.println("Employee Salary:" + emp.getSalary()); - Prints employee salary.
- System.out.println("Employee City:" + emp.getCity()); - Prints employee city.

---

2. ABSTRACTION

Abstraction in Java is the process of hiding internal implementation details and exposing only the essential features to the user. It focuses on what an object does rather than how it does it, achieved using abstract classes and interfaces

- Abstract Classes → Can have abstract methods (no body) and concrete methods (with body).
- Interfaces → Define contracts that classes must implement.

Hiding implementation details and showing only the essential features to the user.

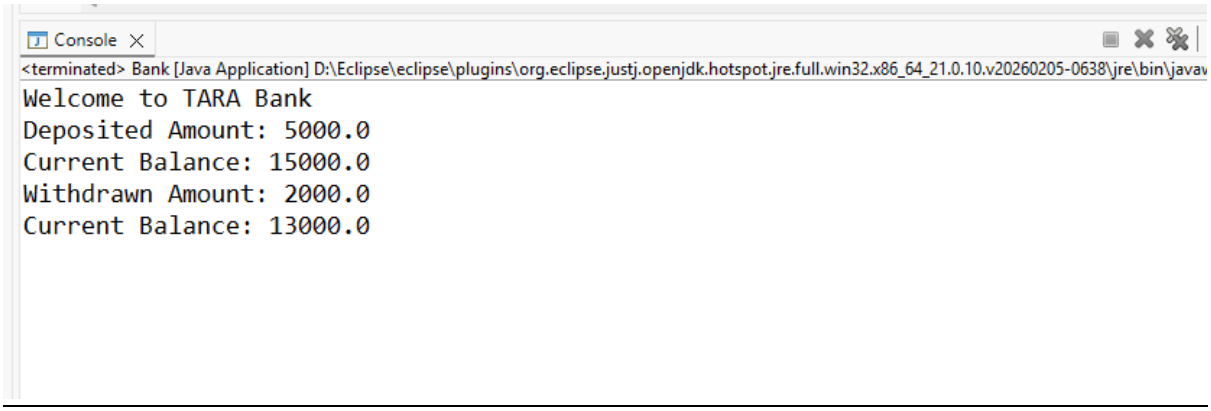
PROGRAM:

```
abstract class Banking {
   abstract void deposit(double amount);
   abstract void withdraw(double amount);

   void Info()
   {
      System.out.println("Welcome to TARA Bank");
   }
}
class SavingsAccount extends Banking
{
   double balance = 10000;

   void deposit(double amount)
   {
      balance = balance + amount;
      System.out.println("Deposited Amount: " + amount);
      System.out.println("Current Balance: " + balance);
   }
   void withdraw(double amount)
   {
      balance = balance - amount;
      System.out.println("Withdrawn Amount: " + amount);
      System.out.println("Current Balance: " + balance);
   }
}
public class Bank {
   public static void main(String[] args)
   {
      SavingsAccount acc = new SavingsAccount();
      acc.Info();
      acc.deposit(5000);
      acc.withdraw(2000);
   }
}
```

OUTPUT:



COMMENTS:

1. Abstract Class Declaration
   - abstract class Banking - Defines an abstract class named Banking.
   - Abstract class - cannot be instantiated directly, but can contain abstract methods and concrete methods.
2. Abstract Methods
   - abstract void deposit(double amount); - Abstract method (no body). Must be implemented by child class.
   - abstract void withdraw(double amount); - Abstract method (no body). Must be implemented by child class.
3. Concrete Method
   - void Info() - Normal method inside abstract class.
   - Prints "Welcome to TARA Bank".
   - Shows that abstract classes can have both abstract and concrete methods.
4. Child Class Declaration
   - class SavingsAccount extends Banking - Child class inherits from Banking.
   - Must provide implementation for abstract methods (deposit, withdraw).
5. Balance Variable
   - double balance = 10000; - Starting balance for savings account.

6. Implementing deposit()

- void deposit(double amount) - Adds money to balance.
- balance = balance + amount; - Updates balance.
- Prints deposited amount and new balance.

7. Implementing withdraw()

- void withdraw(double amount) - Deducts money from balance.
- balance = balance - amount; - Updates balance.
- Prints withdrawn amount and new balance.

8. Main Class

- public class Bank - Defines main class.
- File name must match - Bank.java.

9. Main Method

- public static void main(String[] args) - Entry point of program.

10. Object Creation

- SavingsAccount acc = new SavingsAccount(); - Creates object of child class.
- Even though Banking is abstract, we can use its child class.

11. Method Calls

- acc.Info(); - Calls concrete method from abstract class. Prints welcome message.
- acc.deposit(5000); -- Deposits ₹5000 - balance becomes ₹15000.
- acc.withdraw(2000); - Withdraws ₹2000 - balance becomes ₹13000.